

ECEN 468/719 Advanced Logic Design

Department of Electrical and Computer Engineering

Texas A&M University

Lab 9: Design of System Bus

Objectives

In this lab, we will design a System Bus with Verilog and use Design Analyzer for synthesis. Also, we will use this module to connect SRAM and UART implemented in previous labs.

Introduction

In computer architecture, a bus is a sub-system that transfers data between components inside or between computers. Early computer buses were parallel electrical wires with multiple connections. Now the term is used for any physical arrangement that provides the same logical functionality as a parallel electrical bus. By using a system bus, the number of data pins used to connect all devices can be decreased sharply. It is easy to see in case of the system includes multiple CPUs and memory devices. A typical system bus is shown in [Figure 1](#).

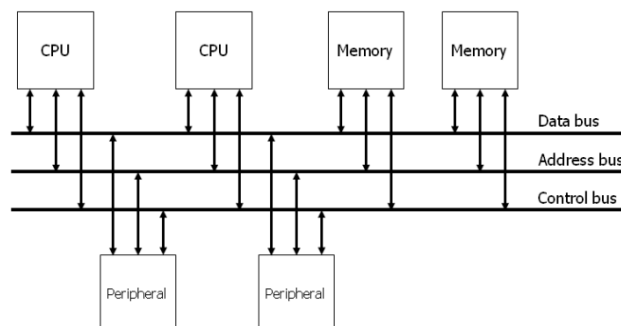


Figure 1. System Bus

A typical system bus consists of a data bus, an address bus, and a control bus, as shown in [Figure 2](#). An address bus is used to specify a physical address. For example, when a processor or DMA-enabled device needs to read or write to a memory location, it specifies that memory location on the address bus (the value to be read or written is sent on the data bus). The width of the address bus determines the size of memory a system can address. A control bus is part of a computer bus used by CPUs for communicating with other devices within the computer. While the address bus carries the information on which device the CPU is communicating with, and the data bus carries the actual data being processed, the control bus carries commands and status signals from the devices.

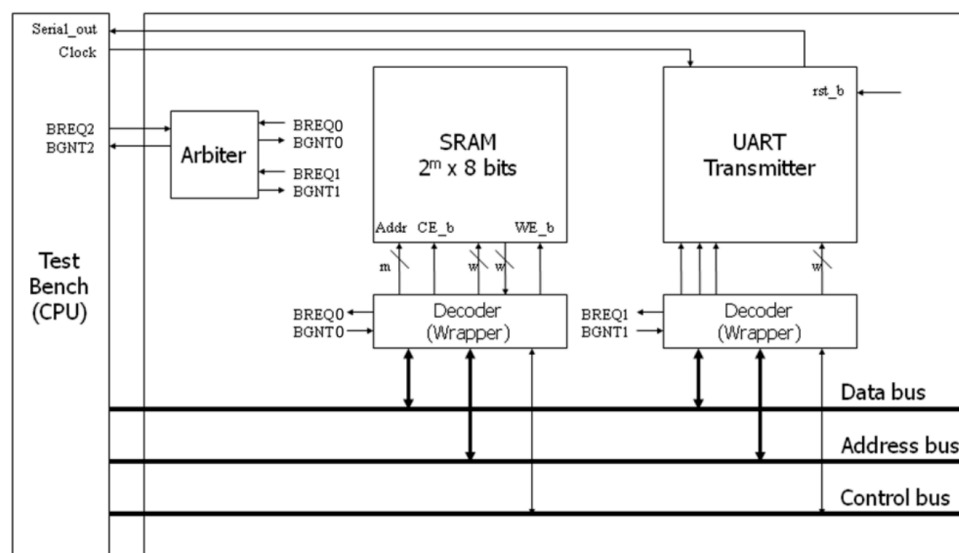


Figure 2. Connections of a system bus

Since the bus is shared amongst all the devices, a method to avoid conflicts must be determined. The method used to determine who gets access to the bus is called **bus arbitration**. The bus arbitration mechanism is designed so that high-priority devices like the processor and RAM get first access to the bus. In contrast, other devices (disks, video cards, sound cards, etc.) get a lower priority and often have to wait until the bus is free. Several modes can be used for arbitration. In this lab, we will use **centralized fixed-priority arbitration**, as shown in [Figure 3](#).

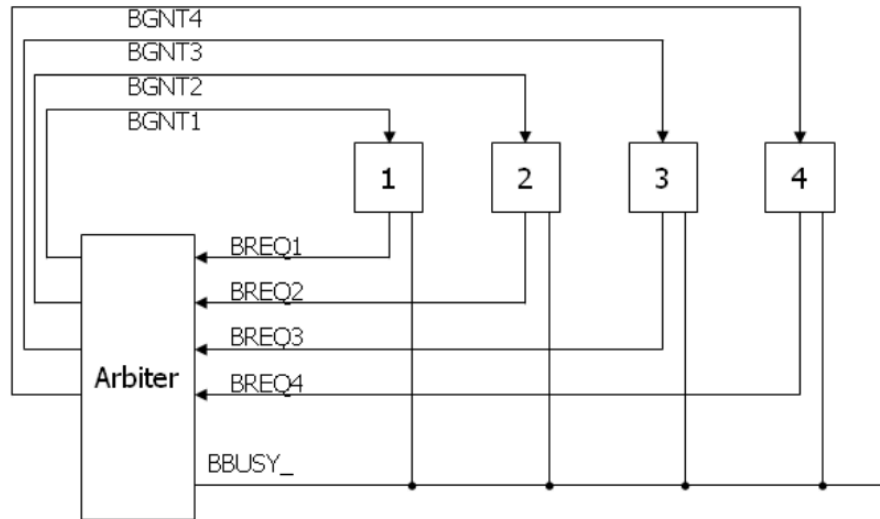


Figure 3. Diagram of the centralized fixed-priority arbitration

In [Figure 3](#), there are three types of signals (BREQ, BGNT, and BBUSY_). The request (**BREQ**) signals are generated by the devices on the bus when they want to use the bus. The request signal is sent to the arbiter, waiting to be granted. If the bus is free and available, the arbiter sends a grant signal (**BGNT**) to the device it grants. If two or more request signals are generated at the same time, the arbiter compares their priorities and sends only one grant signal to the device that has a higher priority. The busy signal (**BBUSY_**) is used to indicate the status of the bus. If the bus is free, the BBUSY_ signal is high (logic 1). If any device gets the right to use the bus, it reads data from the bus or writes data to the bus while pulling the BBUSY_ signal low (logic 0). Once finished, it releases the BBUSY_ signal to high.

[Figure 4](#) shows an example of bus arbitration. (1) master 3 sends the request signal (BREQ3) to the arbiter while master 1 uses the bus. Hence the BBUSY signal is low. (2) The arbiter sends the grant signal (BGNT3). (3) Master 1 completes its operation and releases the BBUSY_ signal (goes high), (4) Master 3 makes BBUSY_ signal active (goes low) and starts to read or write to the bus.

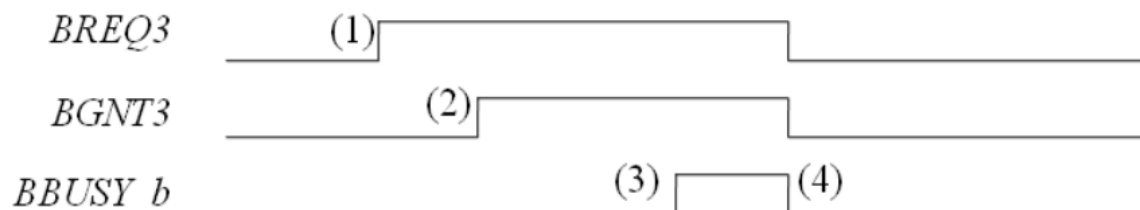


Figure 4. An example of bus arbitration

In our lab, we assume the test bench has the highest priority, and the UART module has the lowest priority. In [Figure 2](#), decoders are attached between SRAM/UART and the system bus. They are responsible for decoding the instructions from the system bus. In this lab, the address bus has 32 bits. The four most significant bits are used for the identification number (ID) of devices. The ID of the CPU (i.e., test bench) in our design is **0011**, the ID of SRAM is **0001**, and the ID of the UART transmitter is **0010**. The remaining bits on the address bus are used for control and address signals. [Figure 5](#) and [Figure 6](#) show the address map to the SRAM and the UART.

ID[31:28]	reserved[27:20]	CE_b[19]	WE_b[18]	Addr[17:0]
-----------	-----------------	----------	----------	------------

Figure 5. Address map to the SRAM

ID[31:28]	reserved[27:4]	T_byte[2]	Byte_ready[1]	Load_XMT_datareg[0]
-----------	----------------	-----------	---------------	---------------------

Figure 6. Address map to the UART

Implementation & Simulation

We will now implement the transmitter part of the UART design.

Please login to the Olympus server and create a working directory for this lab using the following commands.

```
## Create and navigate to the working directory.
mkdir -p $HOME/ECEN468/Lab9/src
cd $HOME/ECEN468/Lab9/src
```

Download the zip file (lab9_code.zip) from Piazza and extract it. In the extracted folders, you will find **Arbiter.v**, **Mainsystem.v**, **WRAP_SRAM.v**, **WRAP_UART.v**, **tb.v**, **SRAM.v**, **generic.sdb**, **osu018_stdcells.v** and **osu018_stdcells.db**. You will design modules in these files after you decompress the file. Copy them to the working directory.

Please copy **Control.v**, **Datapath_Unit.v**, and **UART_XMTR.v** from the previous lab. You will implement your code in **“WRAP_UART.v”** and **“WRAP_SRAM.v”**.

Once you complete the implementation, please verify the correctness of your design. We will

first compile the design using the commands below.

Commands for reference:

```
load-ecen-468 (skip this line on machines in Zach 127)
source /opt/coe/synopsys/vcs-mx/L-2016.06-SP2-4/setup.vcs-mx.sh
vcs -full64 tb.v
```

If it shows compile errors, please recheck your design and fix your code. If the compilation is complete, go to the next step to show the results.

```
./simv
```

Please take screenshots of the terminal output and include them in the report.

To view the graphical waveform of the simulation results, run the following command to invoke WaveView, and open “wave.dump” in WaveView

```
source /opt/coe/synopsys/wv/O-2018.09/setup.wv.sh
wv &
```

The graphical waveform is for debugging. You don’t need to include it in the report.

In this lab, we will synthesize whole modules except SRAM and generate its netlist for gate simulation. The 256K SRAM used for this system is too large to synthesize. In general, memory modules are replaced in a layout process with standard memory modules provided by the company. Thus, we will synthesize all modules except SRAM and connect the module to the netlist after synthesis. You will see **virtSRAM.v**. The module should have exactly the same in and out ports as your SRAM module. We will use this module instead of SRAM.v for synthesis.

Open Mainsystem.v file. And modify `include “SRAM.v” to `include “virtSRAM.v”.

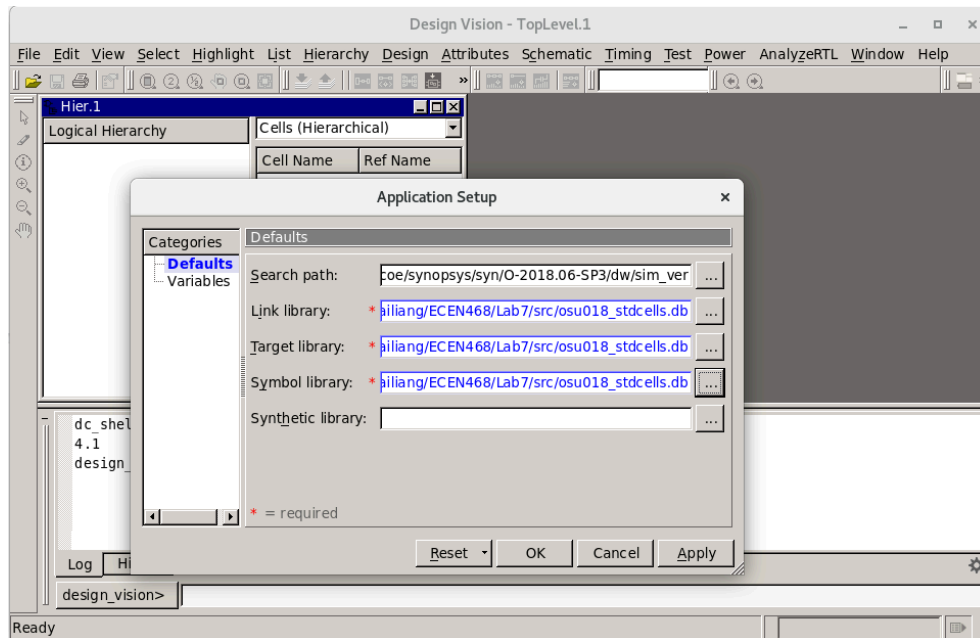
We will use **Design Vision** to synthesize the design.

For your convenience, please invoke Design Vision from a separate directory to isolate the intermediate files generated.

```
## Create and navigate to a separate working directory.  
mkdir $HOME/ECEN468/Lab9/src/dv_Work  
cd $HOME/ECEN468/Lab9/src/dv_Work  
## Launch Design Vision  
source /opt/coe/synopsys/syn/O-2018.06-SP3/setup.syn.sh  
design_vision &
```

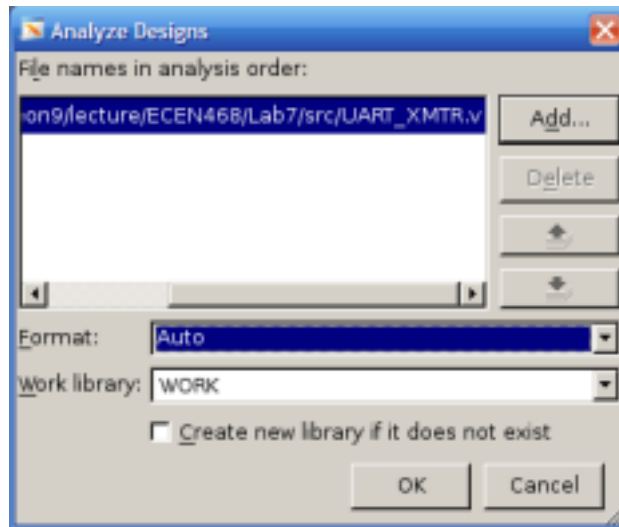
In Design Vision, first, we want to link the libraries.

- Click on File -> Setup -> Defaults
- Add osu018_stdcells.db to “Link Library” (Remove all other library files)
- Add osu018_stdcells.db to “Target Library” (Remove all other library files)
- Add osu018_stdcells.db to “Symbol Library” (Remove all other library files)
- Click on OK.



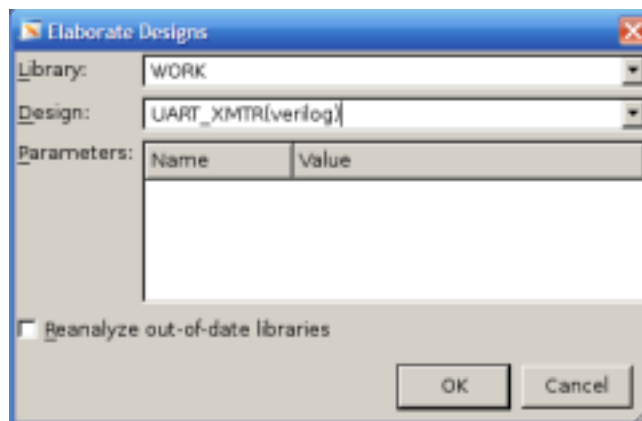
Now we want to *Analyze* the design. In *Analyze*, it reads VHDL or Verilog files, checks for proper syntax and synthesizable logic, and stores the design in some intermediate format.

- Click on File -> Analyze.
- In the pop-up window, add “Mainsystem.v”, and click on OK.



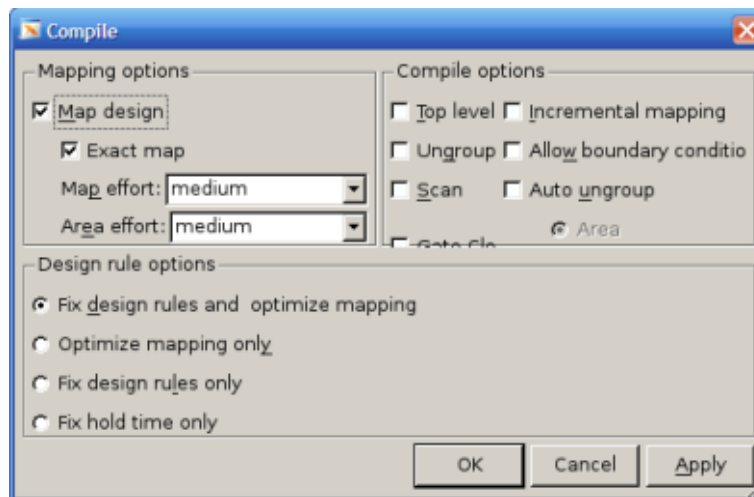
Elaborate Design

- Click on File -> Elaborate.
- In the pop-up window, type “work” in *Library*, and select MAINSYSTEM in *Design*.
- Click on OK.



Synthesize the module

Use ***Design -> Compile Design***. And Click ***OK***.



Save the optimized Verilog netlist.

- Select *TOP* design (MAINSYSTEM)
- File -> Save as
- Please enter "**Mainsystem_gate.v**" as the file name and choose "Verilog" as the file type.
- Check the option "**Save All Designs in Hierarchy**".

Save the Standard Delay Format (SDF) by running the following command in the command window:

```
write_sdf Mainsystem.sdf
```

You may now close Design Vision.

Gate Simulation

a) Copy the Verilog netlist and sdf file to the work folder.

```
cp ./Mainsystem_gate.v ../  
cp ./Mainsystem.sdf ../  
cd ../
```

Open Mainsystem_gate.v file, then you will see the SRAM module which was synthesized with "virtSRAM.v". We will use the original SRAM module in SRAM.v file for gate

simulation. **Delete the SRAM module in Mainsystem_gate.v.**

We will use the previous testbench as a template to create a testbench for the gate simulation.

cp tb.v gate_tb.v

Please open the new testbench “gate_tb.v” and do the following modifications:

- Make sure the following three lines are on the top of the file.

```
`timescale 1ns/10ps
`include "Mainsystem_gate.v"
`include "SRAM.v" // we will use the original SRAM module.
`include "osu018_stdcells.v"
```
- Remove the following line if it exists.

```
`include "UART_XMTR.v"
```
- Make sure the following two lines are at the bottom of the file. (within the module entity)

```
initial
    $sdf_annotate("Mainsystem.sdf", MAINSYSTEM_01);
```
- Change the name of the dump file in the following line in the test bench.

```
$dumpfile ("wave_gate.dump");
```

Run the following command to start the simulation.

```
source /opt/coe/synopsys/vcs-mx/L-2016.06-SP2-4/setup.vcs-mx.sh
vcs -full64 gate_tb.v
```

If you see the error below, please make changes at module WRAP_SRAM and WRAP_UART in Mainsystem_gate.v file.

Err : The 32-bit expression “Addressbus” is connected to 20-bit port “AddressBus”.

- change “tri [19:0] AddressBus” to “tri [31:0] AddressBus”;

Err : The 32-bit expression “Addressbus” is connected to 4-bit port “AddressBus”.

- change “tri [31:28] AddressBus” to “tri [31:0] AddressBus”;

If no error occurs, do the next step.

```
./simv
```

Please take a screenshot of the simulation output, and include it in the report.

Submission

Please only submit one PDF file containing the following items:

Mainsystem:

1. Screenshots of the terminal output after running `./simv`.
2. Justification of the correctness of the results.
3. Screenshots of the code you implemented.

Mainsystem Gate:

1. Screenshots of the simulation output after running `./simv`.
2. Justification of the correctness of the results.